Universidad Autónoma de Tamaulipas
Facultad de Ingeniería Tampico

VERDAD, BELLEZA, PROBIDAD

INGENIERÍA TAMPICO

ASIGNATURA
Programación de interfaces y puertos

6to. Semestre – Grupo "I"

2025 -1

TRABAJO
Desarrollo de Prácticas y Proyectos

UNIDAD
2 - Fundamentos Teóricos sobre interfaces y puertos

Docente: Dr. García Ruiz Alejandro H.

Integrante del Equipo	Nivel de Participación
Rosales Pereles Denisse Ariadna	33.33%
Morales Urrutia Javier Antonio	33.33%
Vicente Bautista Roberto Nicolas	33.33%
Total:	100%

Aniversario 75 UAT 1950-2025

Índice

Repositorio(s) de Proyectos 2

P1. Juego carrusel de Imágenes 2

Descripción del proyecto 2

Introducción 2

Componentes para el desarrollo del proyecto 2

Desarrollo 3

Conclusiones 7

P2. Reloj 24 horas 7

Descripción del proyecto 7

Introducción 8

Componentes para el desarrollo del proyecto 8

Desarrollo 9

Conclusiones 12

Proyecto de la Unidad 2. Recuerda y gana 13

Repositorio(s) de Proyectos

Proyecto	Repositorio
Practica 1	https://github.com/JavierMoralesU/PIP/blob/master/Unidad%202/Ejercicios/PP1.py
Practica 2	https://github.com/JavierMoralesU/PIP/blob/master/Unidad%202/Ejercicios/PP2.py
Proyecto	https://github.com/robertobautistamx/PIP_2025_I_Eq_03/blob/main/Unidad%202/Proyectos/PP3.py

P1. Juego carrusel de Imágenes

Descripción del proyecto

Se realizo un programa que le muestra de forma aleatoria al usuario uno de los textos predeterminados, y el usuario deberá seleccionar la imagen correspondiente utilizando un slider para navegar por las imágenes que son mostradas por medio de su dirección. El programa mostrará una notificación que le dirá al usuario si es que selecciono la imagen incorrecta o si lo hizo de forma correcta en cuyo caso, el enunciado mostrado vuelve a cambiar de forma aleatoria, continuando el juego.

Introducción

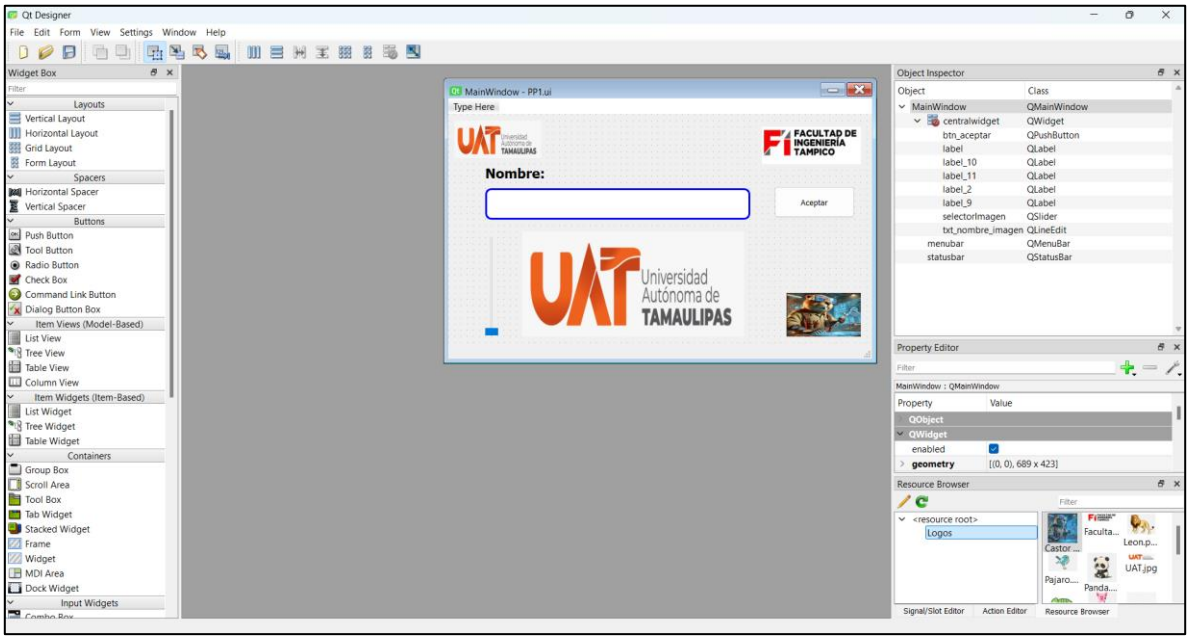
Se desarrollo este proyecto para practicar el uso de imágenes cargadas con ayuda de recurso_rc.qrc que es convertido a recurso_rc.py usando el comando en la terminal: "pyrcc5 recurso_rc.qrc -o recurso_rc.py".
Y también el uso de slider para un cambio de valores de forma mas intuitiva.

Componentes para el desarrollo del proyecto

QLabel: Para mostrar el texto básico dentro de la interfaz y también es a través de una de estas etiquetas que se puede mostrar las imágenes.
QLineEdit: Uno donde se mostrará los enunciados que hacen referencia a las imágenes.
QPushButton: Sera el botón de aceptar para que el usuario confirme que la imagen actual es la correcta o no.
QSlider: Usado para poder cambiar entre las distintas imágenes que se mostraran.

Desarrollo

Diseño en Qt Designer:



```python
PP1.py
'''
import random
from winreg import error

import sys
from PyQt5 import uic, QtWidgets, QtGui
qtCreatorFile = "PP1.ui"  # Nombre del archivo aquí.
Ui_MainWindow, QtBaseClass = uic.loadUiType(qtCreatorFile)
class MyApp(QtWidgets.QMainWindow, Ui_MainWindow):
    def __init__(self):
        QtWidgets.QMainWindow.__init__(self)
        Ui_MainWindow.__init__(self)
        self.setupUi(self)

        self.datosImagenes = \
        {
            0: [":/Logos/UAT.jpg", "UAT"],
            1: [":/Logos/Castor Ingeniero.jpg", "Castor programador"],
            2: [":/Logos/Facultad.jpg", "La facultad de ingenieria..."],
            3: [":/Logos/camaleon.png", "Un camaleon"],
            4: [":/Logos/cerdito.png", "Un cerdito"],
            5: [":/Logos/cocodrilo.png", "Un cocodrilo"],
            6: [":/Logos/delfin.png", "Un delfin"],
            7: [":/Logos/elefante.png", "Un elefante"],
            8: [":/Logos/Leon.png", "Un leon"],
            9: [":/Logos/mono.png", "Un mono"],
            10: [":/Logos/Pajaro.png", "Un pajaro"],
            11: [":/Logos/Panda.png", "Un panda"],
            12: [":/Logos/zorro.png", "Un zorro"],

        }
```

Implementamos el código de Python.

Usamos un diccionario “datosImagenes” para guardar la dirección de las imágenes y una descripción de esta.

Las claves numéricas del diccionario serán las que recorreremos por medio del slider para poder acceder a las imágenes.

```python
        # Área de los Signals
        self.selectorImagen.valueChanged.connect(self.cambiaValor)
        self.selectorImagen.setMinimum(0)
        self.selectorImagen.setMaximum( len(self.datosImagenes) - 1)
        self.selectorImagen.setSingleStep(1)
        self.selectorImagen.setValue(0)

        self.btn_aceptar.clicked.connect(self.Verificar)
        self.cambiaValor()
        self.random()


    # Área de los Slots
    def cambiaValor(self):  2 usages
        valor=self.selectorImagen.value()

        self.ruta_imagen = self.datosImagenes[valor][0]
        self.label_2.setPixmap(QtGui.QPixmap(self.ruta_imagen))
```

Creamos el QSlider “selectorImagen” con un mínimo de 0 y un máximo del tamaño del diccionario menos 1 para poder recorrer todos los elementos del diccionario.

El QSlider se conecta por medio de .ValueChanged.connect() al método cambiaValor. Este método recupera el nuevo valor al que cambio por parte del usuario, y usa este valor para

acceder a la dirección de la imagen por medio de la clave del diccionario, y después esta dirección es usada para cambiar la imagen mostrada en el label, usando QtGui.QPixmap(self.ruta_imagen) para crear un objeto QPixmap y .setPixmap() para mostralo.

```
62      def Verificar(self):  1 usage
63          try:
64              texto = self.txt_nombre_imagen.text()
65              if self.ruta_imagen == texto:
66                  self.msj( title: "Bien",  txt: "Es correcto")
67                  self.random()
68              else:
69                  self.msj( title: "Mal",  txt: "Es incorrecto")
70          except error:
71              print(error)
72
73
74      def random (self):  2 usages
75          n = random.randint ( a: 0, len(self.datosImagenes) - 1)
76          texto = self.datosImagenes[n][0]
77          self.txt_nombre_imagen.setText(texto)
78
79      def msj(self, title, txt):
80          m = QtWidgets.QMessageBox()
81          m.setIcon(QtWidgets.QMessageBox.Information)
82          m.setWindowTitle(title)
83          m.setText(txt)
84          m.exec_()
```

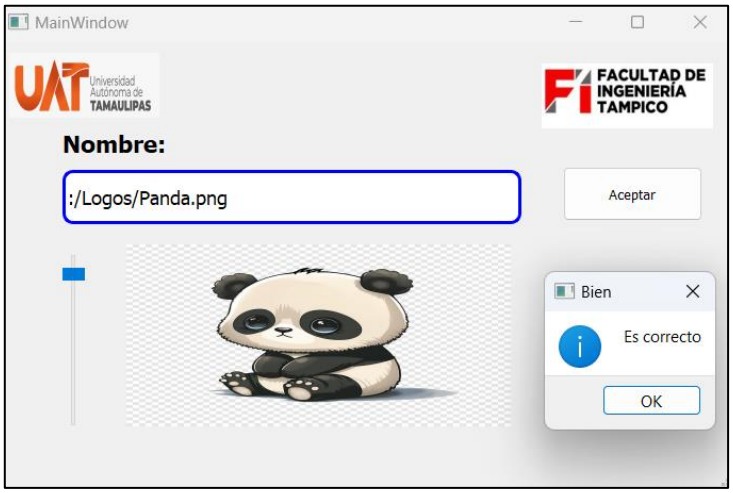
Tenemos un método auxiliar que es msj(), para poder mostrar la ventana emergente (QWidgets.QMessageBox) que informara al usuario si su selección fue correcta o incorrecta.

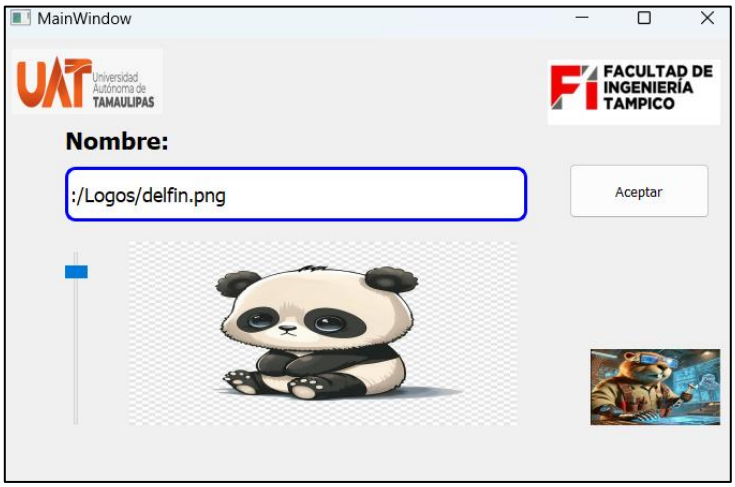
El método random() es el encargado de mostrar un nuevo nombre en el QLineEdit, para ello usa la librería random para generar un entero en el rango de las claves del diccionario y así mostrar la descripción correspondiente.

El método Verificar () es el que compara la descripción mostrada al usuario y que sea el mismo que el del diccionario en la clave escogida atreves del value del QSlider.

Estado base de la aplicación, el nombre mostrado en el lineedit, en este caso es el de la imagen de un Panda, fue decidida aleatoriamente y es la dirección usada para acceder a la imagen.

Este es el mensaje que obtenemos al darle al botón aceptar con una imagen incorrecta.





Al seleccionar la imagen correcta se muestra el mensaje correspondiente y se vuelve a usar el método random() para que aparezca un nuevo enunciado en el nombre.

Conclusiones

La aplicación cumple con el fin de servir como una practica de estos widgets, y sirve como un simple juego de búsqueda de imágenes. El programa puede ser ampliado y modificado para usar más imágenes con relativa personalidad, para ello se seguiría el proceso agregar las nuevas imágenes al archivo .sqr, regenerar el archivo .py, y agregar la ruta al diccionario del código.

P2. Reloj 24 horas

Descripción del proyecto

Se realizo un programa que simula el funcionamiento de un reloj de formato de 24 horas, usando tres QLineEdit para mostrar las horas, minutos y segundos. El reloj empezara a funcionar cuando el usuario presione en el botón que dice “Empezar” y una vez este en funcionamiento se puede volver a oprimir el mismo botón que se encontrara con el texto “Detener” que hará lo indicado.
Mientras que el reloj no se encuentra en funcionamiento, el usuario podrá ingresar la hora, minuto, y segundos que desee de forma manual en el QLineEdit.

Introducción

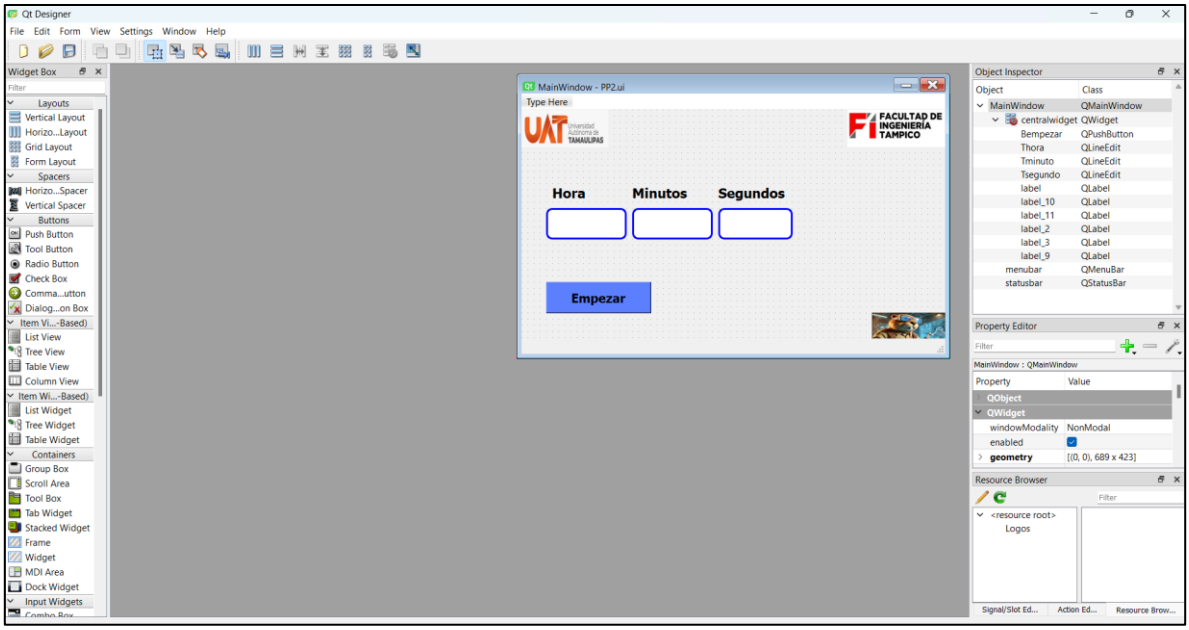
Se desarrollo este proyecto para practicar el uso de imágenes cargadas con ayuda de recurso_rc.qrc que es convertido a recurso_rc.py usando el comando en la terminal: "pyrcc5 recurso_rc.qrc -o recurso_rc.py".
Y también el uso de slider para un cambio de valores de forma mas intuitiva.

Se desarrollo este programa con el objetivo de usar QtCore.QTimer() la cual nos permite ejecutar funciones de forma periódica y de forma asíncrona, es gracias a esta librería que podremos tener un reloj que se actualice cada segundo sin tener que preocuparnos que esto detenga el resto de la aplicación.

Componentes para el desarrollo del proyecto

QLabel: Nos permitirá mostrar la información simple e incambiable al usuario, como los títulos hora, minutos, segundos.
QLineEdit: Sera a través de ellos que mostraremos las correspondientes horas, minutos y segundos que estarán en constante cambio al activar el reloj. Al ser QLineEdits el usuario podrá introducir manualmente la hora que el quiera antes de iniciar el reloj.
QPushBoton: Sera nuestro botón para empezar, una vez llenados los QLineEdit con algún número, podremos presionar el botón para que el reloj avance, y cuando el reloj este avanzando, podremos volver a presionar el botón para detener el reloj en caso de queramos cambiar la hora.

Desarrollo




```python
import random
from winreg import error

import sys
from PyQt5 import uic, QtWidgets, QtGui, QtCore

qtCreatorFile = "PP2.ui"  # Nombre del archivo aquí.
Ui_MainWindow, QtBaseClass = uic.loadUiType(qtCreatorFile)
class MyApp(QtWidgets.QMainWindow, Ui_MainWindow):
    def __init__(self):
        QtWidgets.QMainWindow.__init__(self)
        Ui_MainWindow.__init__(self)
        self.setupUi(self)
        # Área de los Signals

        self.seg = 0
        self.min = 0
        self.hora = 0

        self.Bempezar.clicked.connect(self.iniciarTempo)
        self.timer = QtCore.QTimer()
        self.timer.timeout.connect(self.temporizador)

```

Implementamos el código de Python:

Creamos 3 variables para guardar los segundos, minutos y hora que serán las variables que manejaremos en todo el programa.

Conectamos el botón Bempezar a un método self.iniciarTempo y creamos a timer, un objeto QtCore.QTimer, para conectar su señal timeout al método temporizador.

La señal timeout es una emitida por QtCore.QTimer después de un intervalo de tiempo previamente definido, debido a que el programa cerra un reloj, los intervalos deberán ser cada segundo.

```python
def iniciarTempo(self):  1 usage
    if (self.Bempezar.text() == "Empezar"):
        if (self.Tsegundo.text() != "" and self.Tminuto.text() != "" and self.Thora.text() != ""):
            self.seg = int(self.Tsegundo.text())
            self.min = int(self.Tminuto.text())
            self.hora = int(self.Thora.text())
            self.timer.start(1000)  # Tiempo en milisegundos
            self.Bempezar.setText("Detener")
        else:
            self.msj( title: "Error",  txt: "Faltaron especificar datos")
            if (self.Tsegundo.text() == "" ) :self.Tsegundo.setText("0")
            if (self.Tminuto.text() == "" ) :self.Tminuto.setText("0")
            if (self.Thora.text() == "" ) :self.Thora.setText("0")
    else:
        self.timer.stop()
        self.Bempezar.setText("Empezar")
```

Este es el método de iniciarTempo, el cual se ejecuta al darle clic al botón.

Tiene dos modos: Empezar y Detener.

La parte de empezar requiere que los QLineEdits no estén vacíos (en caso de estarlos los llenara con ceros y mostrara un mensaje de que falto especificar datos) y guardara lo guardado en estos en las variables antes mencionadas, hará timer.star(1000) para que empiecen a funcionar los intervalos y estos sean cada 1000 milisegundos (1 segundo), y se cambia el texto del botón y funcionalidad a Detener.
La función de detener solo ocupa usar timer.stop() para detener el temporizador, y se cambia el texto a Empezar.

```python
def temporizador(self):  1 usage

        self.seg += 1
        self.Tsegundo.setText(str(self.seg))
        if self.seg == 60: #Afecta a segundos y minutos
            self.seg = 0
            self.Tsegundo.setText(str(self.seg))
            self.min += 1
            self.Tminuto.setText(str(self.min))
        if self.min == 60: #Afecta a minutos y horas
            self.min = 0
            self.Tminuto.setText(str(self.min))
            self.hora += 1
            self.Thora.setText(str(self.hora))
        if self.hora == 24:
            self.hora = 0
            self.Thora.setText(str(self.hora))

def msj(self, title, txt):
    m = QtWidgets.QMessageBox()
    m.setIcon(QtWidgets.QMessageBox.Information)
    m.setWindowTitle(title)
    m.setText(txt)
    m.exec_()
```

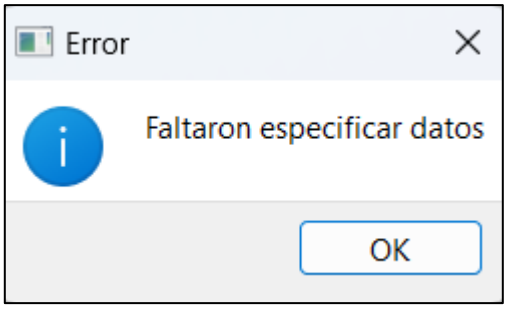
Este es el método temporizador, el que se ejecuta cada intervalo del temporizador, cada segundo.

El método se encarga de actualizar constantemente el texto de los QLineEdits y las variables seg, min, hora.

Se aumenta en 1 la variable seg, y tiene varias condiciones para que al llegar a 60 segundos, los segundos pasen a 0 y se sume 1 a los minutos, 60 minutos aumentan una hora, y 24 horas vuelven a 0.

También contamos con el método msj para poder mostrar una pantalla emergente, en este caso para notificar al usuario cuando no ha establecido todos los valores en los QLineEdits.

Inicio del programa.



Mensaje mostrado al oprimir empezar con los campos de hora, minutos o segundos vacíos

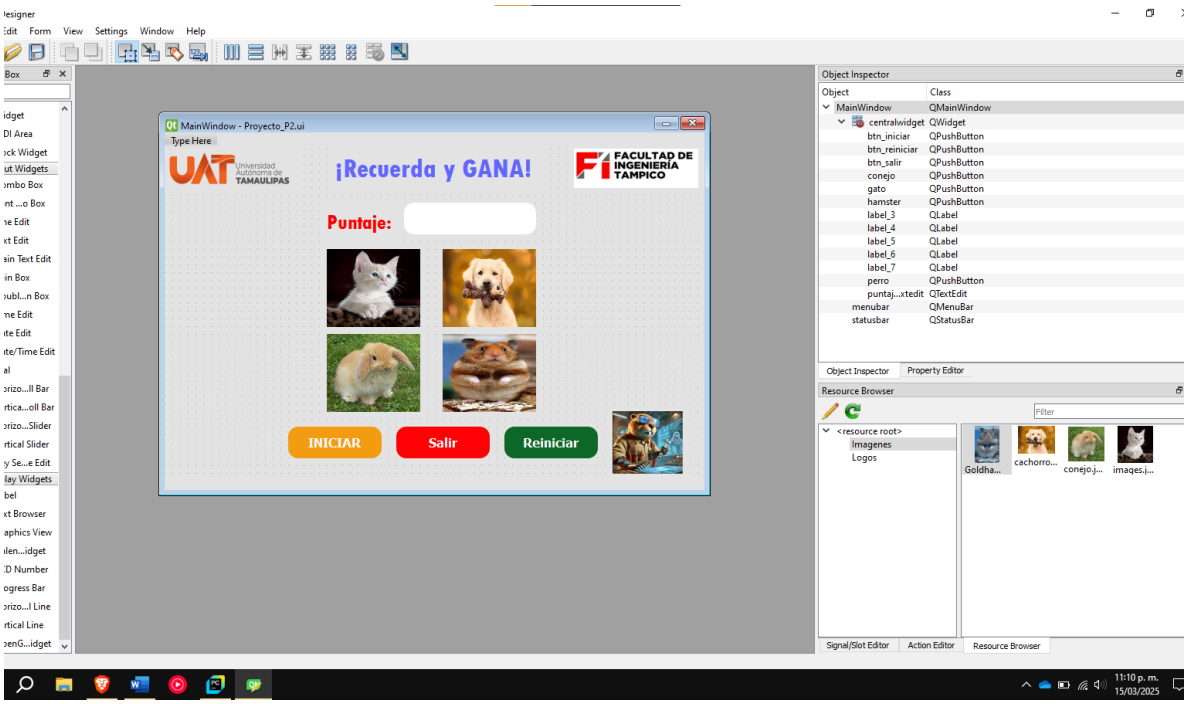
El programa cambiara los números mostrados cada segundo.

Podemos también detener el temporizador para después introducir manualmente la hora que queramos.

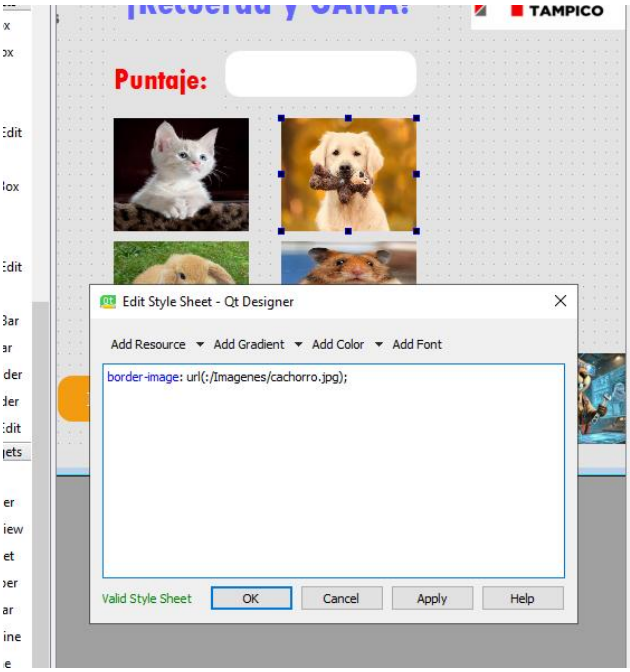
Conclusiones

El programa nos permite simular el funcionamiento de un reloj, nos sirvió para obtener un mejor entendimiento de cómo funcionan los temporizadores de QtCore.QTimer. Este programa podría aun llegar a ampliarse o crear otros programas que también puedan utilizar estas funcionalidades de los temporizadores, como lo pueden ser cronómetros o tareas dinámicas que deban cambiar cada cierto tiempo.

Proyecto de la Unidad 2. Recuerda y gana



Este juego interactivo esta basado en secuencias de imágenes de animales, por lo que el jugador debe memorizar y repetir la secuencia de imágenes mostradas automáticamente.



Este diseño este hecho en “QTDesigner”, lo cual agregamos “QPushButton” para los botones iniciar, salir y reiniciar, asi como también las imágenes en cada botón, es decir, se agregó un “border-image” para que pudiera ver interacción entre la imagen que se muestra, además, usamos “QTextEdit” para mostrar el puntaje del usuario al adivinar la secuencia.

```python
qtCreatorFile = "Proyecto_P2.ui"  # Nombre del archivo aquí.
Ui_MainWindow, QtBaseClass = uic.loadUiType(qtCreatorFile)

class MyApp(QtWidgets.QMainWindow, Ui_MainWindow):  new *
    def __init__(self):  new *
        QtWidgets.QMainWindow.__init__(self)
        Ui_MainWindow.__init__(self)
        self.setupUi(self)

        self.ocultar_imagenes()

        self.secuencia_computadora=[]
        self.secuencia_usuario=[]
        self.puntaje=0
        self.actualizar_puntaje()

        # Botones
        self.btn_iniciar.clicked.connect(self.iniciar)
        self.btn_reiniciar.clicked.connect(self.reiniciar)
        self.btn_salir.clicked.connect(self.salir)

        # Clic en las imágenes
        self.gato.clicked.connect(lambda: self.boton_click("gato"))
        self.perro.clicked.connect(lambda: self.boton_click("perro"))
        self.hamster.clicked.connect(lambda: self.boton_click("hamster"))
        self.conejo.clicked.connect(lambda: self.boton_click("conejo"))
```

Aquí podemos ver, como hacemos el llamado de algunos métodos que se verán mas adelante.
Para este punto, se le da funcionalidad a los botones de iniciar, reiniciar y salir. Asi como también los “clics” en las imágenes. Se usa lambda para que cuando por ejemplo, el usuario de clic en el botón “gato”, se llame a la función boton_clic(“gato”), sin esto, el programa no funcionaria.

```python
    def iniciar(self):  1 usage  new *
        print("JUEGO INICIADO")
        self.secuencia_computadora=[]
        self.secuencia_usuario=[]
        self.nueva_secuencia()

    def reiniciar(self):  new *
        print("Reiniciando juego...")
        self.secuencia_computadora=[]
        self.secuencia_usuario=[]
        self.ocultar_imagenes()
        self.btn_iniciar.setEnabled(True)
        self.puntaje=(0)
        self.actualizar_puntaje()

    def salir(self):  new *
        self.close()

    def ocultar_imagenes(self):  4 usages  new *
        self.gato.setStyleSheet("border-image: none;")  #esconde la imagen
        self.perro.setStyleSheet("border-image: none;")
        self.hamster.setStyleSheet("border-image: none;")
        self.conejo.setStyleSheet("border-image: none;")
```

Se añade los métodos básicos para hacer funcionar el juego. Asi como también los llamados de los métodos que están un poco más abajo.

Para un juego divertido, las imágenes se ocultan para el usuario al momento de adivinar la secuencia.

```python
class MyApp(QtWidgets.QMainWindow, Ui_MainWindow):  new *

    def mostrar_imagen(self, imagen):  1 usage  new *
        if imagen=="gato":
            self.gato.setStyleSheet("border-image: url(:/Imagenes/images.jpeg);")
        elif imagen=="perro":
            self.perro.setStyleSheet("border-image: url(:/Imagenes/cachorro.jpg);")
        elif imagen=="hamster":
            self.hamster.setStyleSheet("border-image: url(:/Imagenes/Goldhamster_terrarium.jpg);")
        elif imagen=="conejo":
            self.conejo.setStyleSheet("border-image: url(:/Imagenes/conejo.jpeg);")

    def nueva_secuencia(self):  2 usages  new *
        self.ocultar_imagenes()
        self.secuencia_computadora.append(self.obtener_imagen_aleatoria()) #imagen aleatoria
        print("Secuencia de la computadora:", self.secuencia_computadora)
        self.mostrar_secuencia_computadora(0) #lo muestra al usuario

    def obtener_imagen_aleatoria(self):  1 usage  new *
        import random
        return random.choice(["gato", "perro", "hamster", "conejo"]) #imagenes existentes en el qt designer
```

Tenemos un método llamado “Mostrar_imagen” y su función es simplemente mostrar la imagen en el botón correspondiente.

Para el método “nueva_secuencia”, lo que hará es generar una nueva secuencia de imágenes aleatorias para que el programa las muestre al usuario. Esto forma parte del juego en el que el usuario debe repetir la secuencia correctamente. Lo cual, en el método “obtener_imagen_aleatoria” selecciona una imagen de forma aleatoria entre las opciones disponibles, en este caso: gato, perro, hámster o conejo. Utiliza el modulo random para elegir uno de estos elementos y lo devuelve.

```python
class MyApp(QtWidgets.QMainWindow, Ui_MainWindow):  new *

    def mostrar_secuencia_computadora(self, index):  2 usages  new *
        if index<len(self.secuencia_computadora):
            imagen=self.secuencia_computadora[index]

            self.mostrar_imagen(imagen)  #mostrar imagen
            QTimer.singleShot(1000, self.ocultar_imagenes)  #1segundo de espera
            QTimer.singleShot(1500, lambda: self.mostrar_secuencia_computadora(index+1)) #sig imagen

        else:
            print("Secuencia completada. Ahora es tu turno.")
            self.secuencia_usuario=[] #reiniciar secuencia
            self.btn_iniciar.setEnabled(False)

    def boton_click(self, imagen):  4 usages  new *
        #validar secuencia del usuario

        if len(self.secuencia_usuario)<len(self.secuencia_computadora):
            self.secuencia_usuario.append(imagen)
            print("Secuencia del usuario:", self.secuencia_usuario)

            #si la secuencia del usuario es correcta hasta ahora
            if self.secuencia_usuario==self.secuencia_computadora[:len(self.secuencia_usuario)]:
                if len(self.secuencia_usuario)==len(self.secuencia_computadora):
                    print("¡Secuencia correcta!")
                    self.secuencia_usuario=[] #reiniciar/repetir
                    self.puntaje+=1  #puntaje incrementa 1+
                    self.actualizar_puntaje()
```

```python
        self.btn_iniciar.setEnabled(False)

    def boton_click(self, imagen):  4 usages  new *
        #validar secuencia del usuario

        if len(self.secuencia_usuario)<len(self.secuencia_computadora):
            self.secuencia_usuario.append(imagen)
            print("Secuencia del usuario:", self.secuencia_usuario)

            #si la secuencia del usuario es correcta hasta ahora
            if self.secuencia_usuario==self.secuencia_computadora[:len(self.secuencia_usuario)]:
                if len(self.secuencia_usuario)==len(self.secuencia_computadora):
                    print("¡Secuencia correcta!")
                    self.secuencia_usuario=[] #reiniciar/repetir
                    self.puntaje+=1  #puntaje incrementa 1+
                    self.actualizar_puntaje()

                    QTimer.singleShot(1000, self.nueva_secuencia)  #break de 1segundos antes de mostyrar la nueva secuencia
            else:
                print("¡Secuencia incorrecta! Perdiste.")
                QtWidgets.QMessageBox.warning(self, title: "Perdiste:(", text: "Rayos, has perdido el juego :(")
                self.reiniciar()

    def actualizar_puntaje(self):  3 usages  new *
        self.puntaje_textedit.setPlainText(f"Puntaje: {self.puntaje}")

if __name__ == "__main__":
```

En este apartado, en el primer método (Mostrar secuencia computadora), lo que hará es mostrar la secuencia de imágenes, una a la vez, con un pequeño retraso en cada imagen.

El objetivo del “boton_click” es simplemente manejar la interacción del usuario al hacer clic en las imágenes, lo cual valida la secuencia correcta y si el usuario acierta, la secuencia se sigue comparando, y si es correcta se aumenta el nivel y se genera una nueva secuencia. Si por alguna razón el usuario falla, reinicia el juego y muestra el mensaje de secuencia incorrecta.

Por último tenemos el método de actualizar el puntaje, este se encarga de actualizar el puntaje en la interfaz del usuario, su objetivo principal es mostrarlo en el Qtextedit llamado puntaje_textedit.

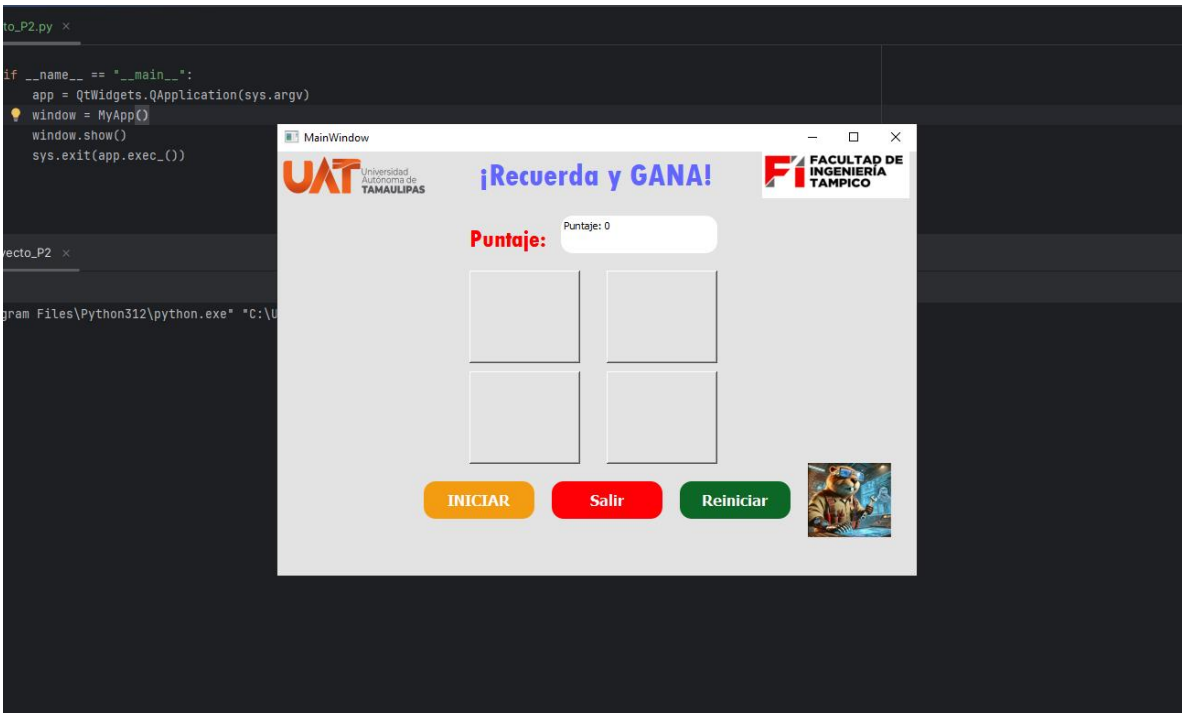
Proyecto_P2.py

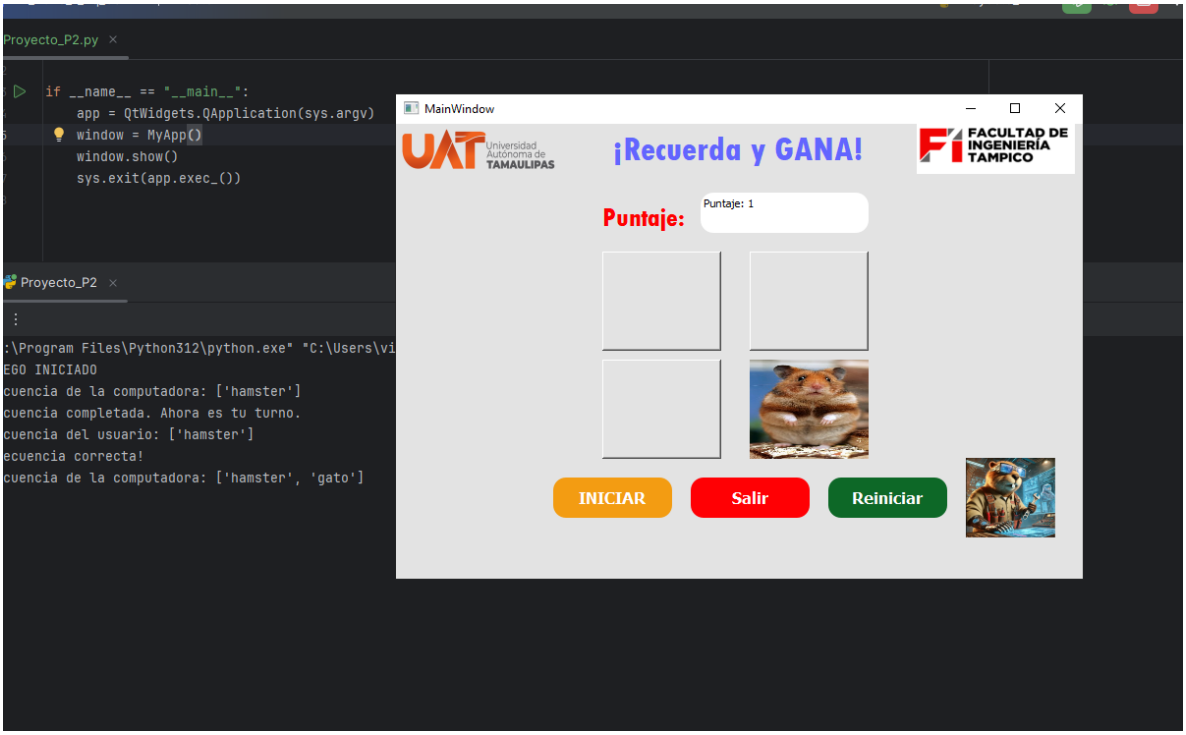
```python
class MyApp(QtWidgets.QMainWindow, Ui_MainWindow):  new *
    def boton_click(self, imagen):  4 usages  new *
            if self.secuencia_usuario==self.secuencia_computadora[:len(self.secuencia_usuario)]:
                if len(self.secuencia_usuario)==len(self.secuencia_computadora):
                    print("¡Secuencia correcta!")
                    self.secuencia_usuario=[] #reiniciar/repetir
                    self.puntaje+=1  #puntaje incrementa 1+
                    self.actualizar_puntaje()

                    QTimer.singleShot(1000, self.nueva_secuencia)  #break de 1segundos antes de mostyrar la nueva secuencia
            else:
                print("¡Secuencia incorrecta! Perdiste.")
                QtWidgets.QMessageBox.warning(self, title: "Perdiste:(", text: "Rayos, has perdido el juego :(")
                self.reiniciar()

    def actualizar_puntaje(self):  3 usages  new *
        self.puntaje_textedit.setPlainText(f"Puntaje: {self.puntaje}")

if __name__ == "__main__":
    app = QtWidgets.QApplication(sys.argv)
    window = MyApp()
    window.show()
    sys.exit(app.exec_())
```

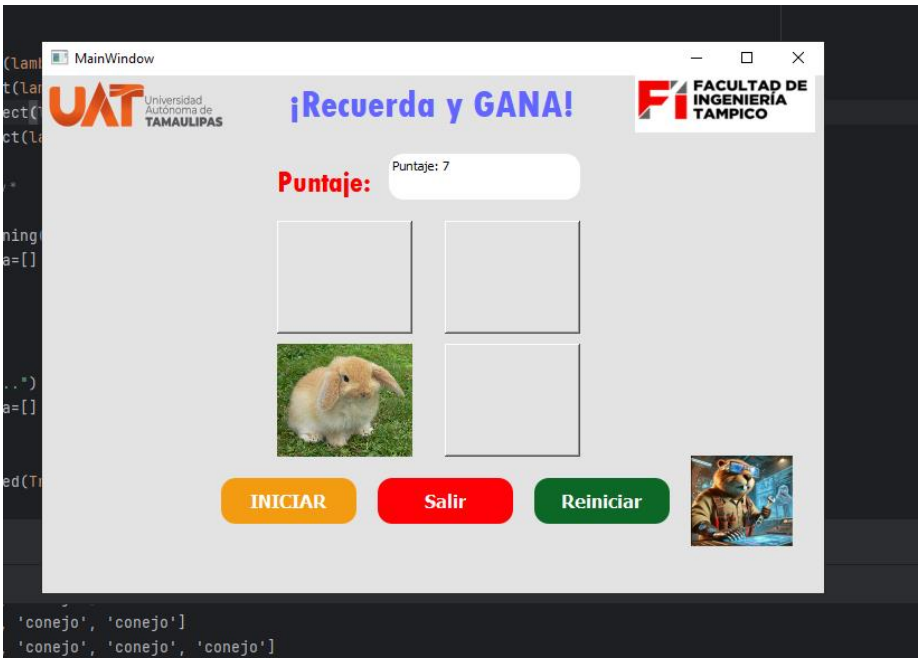

to_P2.py
if __name__ == "__main__":
app = QtWidgets.QApplication(sys.argv)
window = MyApp()
window.show()
sys.exit(app.exec_())
vecto_P2
gram Files\Python312\python.exe" "C:\U
MainWindow
¡Recuerda y GANA!
FACULTAD DE INGENIERÍA TAMPICO
Puntaje:
Puntaje: 0
INICIAR
Salir
Reiniciar


Proyecto_P2.py
if __name__ == "__main__":
app = QtWidgets.QApplication(sys.argv)
window = MyApp()
window.show()
sys.exit(app.exec_())
Proyecto_P2
:\Program Files\Python312\python.exe" "C:\Users\vi
EGO INICIADO
cuencia de la computadora: ['hamster']
cuencia completada. Ahora es tu turno.
cuencia del usuario: ['hamster']
ecuencia correcta!
cuencia de la computadora: ['hamster', 'gato']
MainWindow
¡Recuerda y GANA!
FACULTAD DE INGENIERÍA TAMPICO
Puntaje:
Puntaje: 1
INICIAR
Salir
Reiniciar

MainWindow
Universidad Autónoma de TAMAULIPAS
¡Recuerda y GANA!
FACULTAD DE INGENIERÍA TAMPICO
Puntaje:
Puntaje: 2
Perdiste:(
Rayos, has perdido el juego :(
OK
INICIAR
Salir
Reiniciar

MainWindow
Universidad Autónoma de TAMAULIPAS
¡Recuerda y GANA!
FACULTAD DE INGENIERÍA TAMPICO
Puntaje:
Puntaje: 0
BIEN
JUEGO REINICIADO
OK
INICIAR
Salir
Reiniciar

MainWindow
¡Recuerda y GANA!
Puntaje:
Puntaje: 0
Recuerda y GANA
Juego comienza en 3 ... 2 ... 1 ... GO
OK
INICIAR
Salir
Reiniciar
"C:\Users\vicen\Desktop\PIP_2025_1_Eq_03\Unidad 2\Proyectos\Proyecto_P2.py"


MainWindow
¡Recuerda y GANA!
Puntaje:
Puntaje: 7
INICIAR
Salir
Reiniciar
'conejo', 'conejo']
'conejo', 'conejo', 'conejo']